MOSAI: A Link Energy and Cycles Co-Optimization Simulation Framework for NoC-based DNN/LLM Accelerators - Hands-on Tutorial

1. Project Overview

MOSAIC is a Network-on-Chip (NoC) simulator supporting two workloads:

CNN inference
LLM Attention Matrix computation

Project Structure

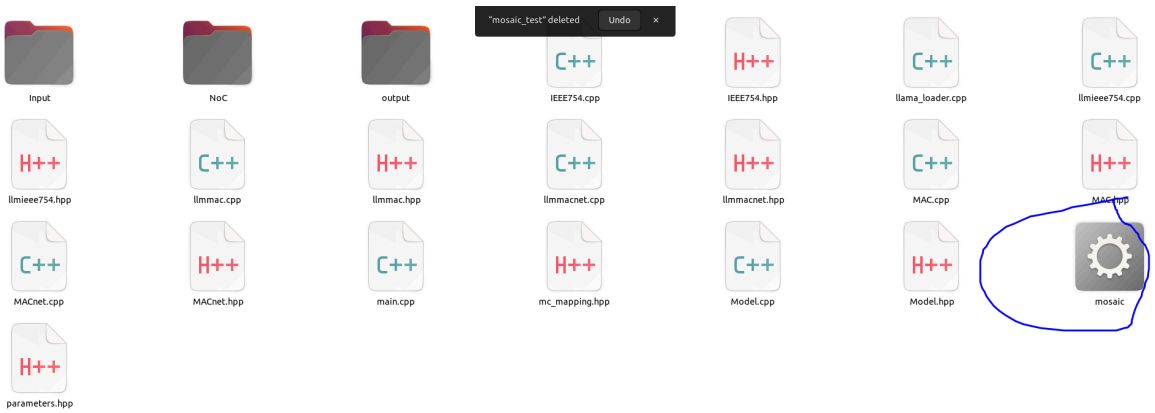
```
MOSAIC/
├── src/
│   ├── main.cpp          # Main entry (to start the NoC and AI workload)
│   ├── parameters.hpp    # Core configuration file
│   ├── NoC/              # NoC components: Router, NI, Flit, etc. (communication part)
│   ├── Input/            # Input data files
│   │   ├── llminput/     # LLM inputs
│   │   └── *.txt         # CNN  inputs
│   ├── output/           # Simulation output directory
│   └──MAC/LLMMAC .hpp .cpp          # Processing elements (computation part)
│   └──IEEE754/LLMIEEE754 .hpp .cpp        bit-level representative of floating-point data
|                                          and “1”-bit count-based ordering.
└── readme.md
```

2. Environment Setup and Compilation

OS: Linux (Ubuntu 20.04+ recommended)
Compiler: GCC/G++ with C++11 support
IDE (optional): Eclipse IDE for C/C++

2.1 Compile with dthe efault configuration

```bash
pwd
ls
# you should be at the root folder and  see src/ readme  etc
# Compile all source files
g++ -std=c++11 -O2 -o src/mosaic src/*.cpp src/NoC/*.cpp
```



You will see the executable file.

```bash
./mosaic
```

Some output at the begining

```
 $ ./mosaic
Initialize LLM Attention Simulation
Creating VCNetwork with 64 routers
Object is being created
Creating LLMMACnet with 64 MAC units
204LLMMAC[0] at (0,0) -> MC 17
204LLMMAC[1] at (0,1) -> MC 17
204LLMMAC[2] at (0,2) -> MC 19
204LLMMAC[3] at (0,3) -> MC 19
204LLMMAC[4] at (0,4) -> MC 21
204LLMMAC[5] at (0,5) -> MC 21
204LLMMAC[6] at (0,6) -> MC 23
204LLMMAC[7] at (0,7) -> MC 23
204LLMMAC[8] at (1,0) -> MC 17
204LLMMAC[9] at (1,1) -> MC 17
204LLMMAC[10] at (1,2) -> MC 19
204LLMMAC[11] at (1,3) -> MC 19
204LLMMAC[12] at (1,4) -> MC 21
204LLMMAC[13] at (1,5) -> MC 21
204LLMMAC[14] at (1,6) -> MC 23
204LLMMAC[15] at (1,7) -> MC 23
Successfully loaded X_input matrix (8x4096)
Successfully loaded Wq matrix (128x4096)
Loaded real matrices from ../src/Input/llminput/
First 5 values from X_input[0]: 0.0248357 -0.00276529 0.0323844 0.0304606 -0
Q_resOutput_matrix after initialization (first 3x3):
```

Some results for checking the computation correctness:

```
First 5x5 elements of attention_output_table:
-----------------------------------------------
          [0]         [1]         [2]         [3]         [4]
[0]      0.015450    -0.011194     0.003365     0.038288    -0.002731   ...
[1]     -0.009323     0.002241     0.000770     0.010050     0.013300   ...
[2]      0.004713    -0.051633     0.060124     0.041851     0.049868   ...
[3]     -0.050703    -0.001162     0.007682    -0.025303     0.020468   ...
[4]     -0.002006    -0.022041     0.020444     0.036356    -0.001902   ...
...   (showing first 5 of 8 rows)
```

And the most importance results: cycles (inference latency) and the bit transitons/flips.

```
=== NETWORK STATISTICS ===
Core Metrics:
  Total Cycles: 108283
  Total Flits Created: 590848
  Total Hop Count (Router+NI): 1606368
  Router Hop Count: 1015520
 mainauthorRouterZeroBTHopTotalCount  110936
  NI Hop Count: 590848
  Total Bit Flips (Router-only): 187981955
 reqRouterFlip 28255 respRouterFlip 187924482 resRouterFlip 29218
 reqRouterHop 112640 respRouterHop 901120 resRouterHop 1760
  Average Hops per Flit (total): 2.72
  Average Router Hops per Flit: 1.72
  Average NI Hops per Flit: 1.00
  Average Bit Flips per Flit (totalhops) : 318.16
  Average Bit Flips per Router Hop: 185.11
  Average Bit Flips per respRouter Hop: 208
```

3. Configuration Options
3.1 To reproduce the paper's work, we need to
```cpp
//1. Change the parameter.hpp
//2. Compile again, "g++ -std=c++11 -O2 -o mosaic src/*.cpp src/NoC/*.cpp", see section 2.1
//3.run the new executable file
```
3.1 LLM attention matrix configuration
```cpp
#define AUTHORLLMSwitchON
#define fulleval // this mode will enable read saved  realistic input. // #define randomeval should be commented, unless for fast-test purpose.
```
select one NoC size, keep others commented
```cpp
//#define NOCSIZEMC2_4X4      // 2 MCs in 4x4 mesh (base tile pattern)
#define NOCSIZEMC8_8X8      // 8 MCs in 8x8 mesh (2x2 tiles)
//#define NOCSIZEMC32_16X16   // 32 MCs in 16x16 mesh (4x4 tiles)
//#define NOCSIZEMC128_32X32  // 128 MCs in 32x32 mesh (8x8 tiles)
```

Select one token size, keep others commented
```cpp
#define LLM_TOKEN_SIZE 1  // 8tokesn  ~50 seconds on Intel 10700.
//#define LLM_TOKEN_SIZE 2    //128tokens . This takes more than ~20minutes.
```

Select one case to be tested.
```cpp
// Test Case Configuration - Choose one
#define case1_default
//#define case2_samos
//#define case3_affiliatedordering
//#define case4_seperratedordering
//#define case5_COMBO1
//#define case6_COMBO2
//#define case7_FireAdvance
//#define case8_BinarySwitch
//#define case9_MOSAIC1
//#define case10_MOSAIC2
```
3.2 CNN attention matrix configuration

comment the LLM switch and comment the LLM token setting.
```cpp
//#define AUTHORLLMSwitchON
//#define LLM_TOKEN_SIZE 1  // 8tokesn  ~50 seconds on Intel 10700.
//#define LLM_TOKEN_SIZE 2    //128tokens . This takes more than ~20minutes.
```

Select one case to be tested.
```cpp
// Test Case Configuration - Choose one
#define case1_default
//#define case2_samos
//#define case3_affiliatedordering
//#define case4_seperratedordering
//#define case5_COMBO1
//#define case6_COMBO2
//#define case7_FireAdvance
//#define case8_BinarySwitch
//#define case9_MOSAIC1
//#define case10_MOSAIC2
```

3.3
```bash
# compile
g++ -std=c++11 -O2 -o mosaic src/*.cpp src/NoC/*.cpp
```

```
# run
./mosaic
```

Enjoy!